value, you can use Google's `protobuf` library—which is included in `akka-projection-eventsourced` as a dependency—as follows:

```
import com.google.protobuf.any.{ Any => PbAny }

class MarketProjectionHandler ... {

  def serialize(event: Market.Event): Array[Byte] = {
      val proto = ...
      PbAny.pack(proto, "market-projection").toByteArray
  }
}
```

To use this, you need a `proto` object that you can create with the values from the `Market.Event`. The definition of the `proto` objects corresponds to the events `MarketClosed`, `MarketOpened`, and `MarketCancelled`:

```
syntax = "proto3";

option java_package = "betting.house.projection.proto";

message MarketClosed {
      string marketId = 1;
      int32 result = 2;
}
message MarketOpened {
      string marketId = 1;
}
message MarketCancelled {
      string marketId = 1;
      string reason = 2;
}
```

With these definitions and the automatically generated `proto` objects, you can pack the market events into an `Array[Byte]` like this: